Programming Principles I

Week 6 · Class 11

Characters and strings

One character · a character is a number · text as an array of char

```
char name[] = "Leyla";
printf("Hello, %s!\n", name);
```

Quiz 3 is this Friday

Fri 16 October, the first 25 minutes of class.

It covers classes 7 to 10: functions, scope, recursion,

arrays, search, min and max, bubble sort.

Paper and pen. No laptop, no phone.

Prepare with Knowledge checks 7 to 10 and the practice tests.

What Friday gave us

1. Send an array **and its size**: `f(a, n)`
2. A function can **change** the caller's array
3. Search: return the index, or `-1` **after** the loop
4. Min and max start from `a[0]`; bubble sort swaps neighbours

Recap: where does it stop?

PREDICT

There is no `i < 6` here.

What stops the loop?

What are the two lines?

```
int a[6] = {4, 8, 15, 16, 23, 0};
int i = 0;

while (a[i] != 0) {
    printf("%d ", a[i]);
    i++;
}
printf("\ncount: %d\n", i);
```

Recap: where does it stop?

PREDICT

There is no `i < 6` here.

What stops the loop?

What are the two lines?

```
int a[6] = {4, 8, 15, 16, 23, 0};
int i = 0;

while (a[i] != 0) {
    printf("%d ", a[i]);
    i++;
}
printf("\ncount: %d\n", i);
```

ANSWER

`4 8 15 16 23`
then `count: 5`.
The `0` marks the end.

Today's roadmap

1. One **character**: the type `char`
2. A character **is a number**
3. Testing characters: `<ctype.h>`
4. A **string**: an array of `char` with an end mark
5. Reading strings: `scanf` and `fgets`
6. Loops over a string

> Today's recap loop comes back at the end, with letters instead of numbers.

1 One character

The type char

A char holds one character

> **A letter, a digit, a space, a sign: one of them.**

Written in **single** quotes: `'A'`, `'7'`, `' '`, `'?'`

Printed with `%c`.

You met `char` in class 3. Today it gets a job.

Three characters

```c
char grade = 'A';
char digit = '7';
char space = ' ';

printf("[%c] [%c] [%c]\n", grade, digit, space);
```

Output

```
[A] [7] [ ]
```

`'7'` is the **character** 7.

The space is a character too.

A char is a small number

> **Inside the computer, every character is a code.**

`'A'` is stored as 65. `'a'` as 97. `'0'` as 48.

The table of codes is called **ASCII**.

`%c` shows the character, `%d` shows its code.

The same value, two ways

```
printf("%c = %d\n", 'A', 'A');
printf("%c = %d\n", 'B', 'B');
printf("%c = %d\n", 'a', 'a');
printf("%c = %d\n", '0', '0');
printf("%c = %d\n", '9', '9');
printf("[%c] = %d\n", ' ', ' ');
```

Output

```
A = 65
B = 66
a = 97
0 = 48
9 = 57
[ ] = 32
```

> %c then %d.

Codes to remember

characters	codes	in order?
`'A'` to `'Z'`	65 to 90	yes
`'a'` to `'z'`	97 to 122	yes
`'0'` to `'9'`	48 to 57	yes
space	32	

You do not learn the numbers. You use the **order**: letters and digits come one after another.

A loop over the alphabet

```
for (char c = 'A'; c <= 'Z'; c++) {
    printf("%c", c);
}
printf("\n");
```

Output

```
ABCDEFGHIJKLMNOPQRSTUVWXYZ
```

c++ adds 1 to the code: `'A'` (65) becomes `'B'` (66). The loop stops after `'Z'`.

2 A character is a number

Arithmetic on characters

Predict: letters plus numbers

```
char c = 'a';
printf("%c\n", c + 1);
printf("%d\n", c + 1);
printf("%c\n", 'A' + 25);

char x = 'e';
printf("%c: letter %d\n", x, x - 'a' + 1);
```

'a' is 97.

What are the four lines?

Letters plus numbers

```
char c = 'a';
printf("%c\n", c + 1);
printf("%d\n", c + 1);
printf("%c\n", 'A' + 25);

char x = 'e';
printf("%c: letter %d\n", x, x - 'a' + 1);
```

Output

```
b
98
Z
e: letter 5
```

Same value 98: %c gives b.

The place of a letter

> **`letter - 'a'` is its place, counting from 0.**

`'a' - 'a'` is 0. `'b' - 'a'` is 1.

`'e' - 'a'` is 101 - 97 = 4.

Add 1 to count from 1: e is letter 5.

A small letter becomes a capital

```
char c = 'g';
char big = c - 'a' + 'A';

printf("%c becomes %c\n", c, big);
printf("%d becomes %d\n", c, big);
```

Output

```
g becomes G
103 becomes 71
```

Place of g: 6. Plus `'A'`: `'G'`.

Always 32 apart.

Predict: the character 7

PREDICT

`d` holds the character `'7'`.

What are the three numbers?

```
char d = '7';

printf("%d\n", d);
printf("%d\n", d - '0');
printf("%d\n", (d - '0') * 2);
```

The character 7 is not the number 7

```
char d = '7';

printf("%d\n", d);
printf("%d\n", d - '0');
printf("%d\n", (d - '0') * 2);
```

Output

```
55
7
14
```

`'7'` has the code 55.

`d - '0'`: 55 - 48 = 7.

A digit and its value

The value of a digit character is `c - '0'`.

`'0' - '0'` is 0, `'7' - '0'` is 7, `'9' - '0'` is 9.

Now it is a number: you can add it, multiply it.

Class 12 turns a whole string like `"2026"` into a number.

Careful: which quotes?

CAREFUL

One letter in **double** quotes.

What does gcc say?

```
char c = "A";
printf("%c\n", c);
```

Double quotes: gcc refuses

```c
char c = "A";
printf("%c\n", c);
```

gcc -Wall says

```
error: initialization
of 'char' from
'char *' makes integer
from pointer without
a cast
```

`"A"` is a **string**, not one character. A `char` takes `'A'`, in single quotes.

'A' or "A"?

'A' single	"A" double
one **character**	a **string**
the number 65	two boxes: `A` and the end mark
goes in a `char`	goes in a `char` array

> The end mark is the next big idea of today.

3 Testing characters

Is it a letter? A digit?

Which kind of character?

```
scanf(" %c", &c);

if (c >= 'a' && c <= 'z') {
    printf("%c is a small letter\n", c);
} else if (c >= 'A' && c <= 'Z') {
    printf("%c is a capital letter\n", c);
} else if (c >= '0' && c <= '9') {
    printf("%c is a digit\n", c);
} else {
    printf("%c is something else\n", c);
}
```

Run, input Q

```
Type one character: Q
Q is a capital letter
```

<ctype.h> does it for you

Add `#include <ctype.h>` at the top.

`isdigit(c)`, `isalpha(c)`, `isupper(c)`, `islower(c)`: yes or no.

`toupper(c)`, `tolower(c)`: the letter in the other case.

Not a letter? `toupper` gives it back unchanged.

The six functions

call	question or job	example
`isdigit(c)`	a digit?	`'7'`: yes
`isalpha(c)`	a letter?	`'?'`: no
`isupper(c)`	a capital?	`'Q'`: yes
`islower(c)`	a small letter?	`'Q'`: no
`toupper(c)`	make it a capital	`'g'` gives `'G'`
`tolower(c)`	make it small	`'Q'` gives `'q'`

ctype.h at work

```
printf("%c %c\n", toupper('g'), tolower('Q'));
printf("%c %c\n", toupper('G'), toupper('7'));

if (isdigit('7')) {
    printf("7 is a digit\n");
}
if (isupper('Q')) {
    printf("Q is a capital\n");
}
if (!isalpha('?')) {
    printf("? is not a letter\n");
}
```

Output

```
G q
G 7
7 is a digit
Q is a capital
? is not a letter
```

`toupper('7')`: still 7.

Careful: what is "yes"?

CAREFUL

Three tests, printed with %d.

What are the three numbers?

```
printf("%d\n", isdigit('7'));
printf("%d\n", isalpha('x'));
printf("%d\n", isdigit('x'));
```

"Yes" is not always 1

```
printf("%d\n", isdigit('7'));
printf("%d\n", isalpha('x'));
printf("%d\n", isdigit('x'));
```

Output

```
1
2
0
```

`isalpha` said 2. Yes is "not 0".

Write `if (isdigit(c))`. Never `if (isdigit(c) == 1)`: it is false when the answer is 2.

The same test, with ctype.h

```
if (isupper(c)) {
    printf("capital letter\n");
} else if (islower(c)) {
    printf("small, capital %c\n", toupper(c));
} else if (isdigit(c)) {
    printf("digit, value %d\n", c - '0');
} else {
    printf("something else\n");
}
```

Run, input g

```
Type one character: g
small, capital G
```

4 Strings

Text as an array of char

A string is an array of char

> **The characters, one per box, then an end mark.**

The end mark is `'\0'`, the **null character**. Its code is 0.

A string in double quotes gets the `'\0'` for free.

Like the `0` at the end of today's recap array.

"hello" in memory

index	0	1	2	3	4	5
box	h	e	l	l	o	\0

5 letters, **6** boxes. The last box holds `'\0'`.

Two ways to make a string

```
char w[6] = {'h', 'e', 'l', 'l', 'o', '\0'};
char s[] = "hello";

printf("%s\n", w);
printf("%s\n", s);
```

Output

```
hello
hello
```

`%s` prints the characters, box by box, until it meets `'\0'`.

Predict: how many boxes?

PREDICT

`"hi"` is in the array.

What are the two lines?

```
char s[] = "hi";

printf("%c %c\n", s[0], s[1]);
printf("%d %d %d\n", s[0], s[1], s[2]);
```

"hi" takes three boxes

```
char s[] = "hi";

printf("%c %c\n", s[0], s[1]);
printf("%d %d %d\n", s[0], s[1], s[2]);
```

Output

```
h i
104 105 0
```

`s[2]` is `'\0'`: code 0.

Count the end mark

A string of n letters needs n + 1 boxes.

`"hi"`: 3 boxes. `"hello"`: 6 boxes.

`char s[] = "hello";` gcc counts for you.

`char name[20];` room for 19 letters and the `'\0'`.

Careful: three boxes for "cat"

CAREFUL

`word` has 3 boxes.

`"cat"` needs 4.

What is printed?

```
char next[] = "dog";
char word[3] = "cat";

printf("%s\n", word);
printf("%s\n", next);
```

No end mark: %s goes on

```
char next[] = "dog";
char word[3] = "cat";

printf("%s\n", word);
printf("%s\n", next);
```

Output

```
catdog
dog
```

No `'\0'` after `t`.

`%s` walked into `next`.

`gcc -Wall` says nothing. `gcc -Wall -Wextra` warns. Another computer can print something else.

Printing a string

```
char name[] = "Leyla";

printf("Hello, %s!\n", name);
printf("First letter: %c\n", name[0]);
```

Output

```
Hello, Leyla!
First letter: L
```

`%s` and `name`: all of it.

`%c` and `name[0]`: one box.

Change one box

```c
char word[] = "cat";
printf("%s\n", word);

word[0] = 'b';
printf("%s\n", word);

word[2] = 'r';
printf("%s\n", word);
```

Output

```
cat
bat
bar
```

A string is an array: change `word[i]`.

Careful: no = for a whole string

```
char word[] = "cat";
word = "dog";
printf("%s\n", word);
```

gcc -Wall says

```
error: assignment to
expression with
array type
```

Class 9: `b = a;` did not compile for arrays. A string is an array. **Class 12** gives you a library function that copies.

5 Reading strings

scanf and fgets

Read a word with scanf

```c
char name[20];
printf("Your name: ");
scanf("%s", name);
printf("Hello, %s!\n", name);
```

Run, input Leyla

```
Your name: Leyla
Hello, Leyla!
```

> `%s`, and **no** `&`.

Why no &?

> **The name of an array already says where it is.**

`scanf("%d", &n);` an `int` needs the `&`.

`scanf("%s", name);` an array does not.

Why? Pointers explain it, after the midterm.

Predict: a full name

PREDICT

The program is the same.

You type `Leyla Aliyeva`.

What is printed?

```
char name[20];
printf("Your name: ");
scanf("%s", name);
printf("Hello, %s!\n", name);
```

scanf stops at a space

```c
char name[20];
printf("Your name: ");
scanf("%s", name);
printf("Hello, %s!\n", name);
```

Run, input Leyla Aliyeva

```
Your name: Leyla Aliyeva
Hello, Leyla!
```

`%s` reads **one word**.

It stops at a space or Enter.

Careful: the & gets a warning

```
char name[20];
printf("Your name: ");
scanf("%s", &name);
printf("Hello, %s!\n", name);
```

gcc -Wall says

```
warning: format '%s'
expects argument of
type 'char *', but
argument 2 has type
'char (*)[20]'
```

Delete the &. Read warnings: this one names the argument.

Read a whole line: fgets

```
char line[50];
printf("Your full name: ");
fgets(line, 50, stdin);
printf("<%s>\n", line);
```

Run, input Leyla Aliyeva

```
Your full name: Leyla Aliyeva
<Leyla Aliyeva
>
```

`fgets(line, 50, stdin)`: where, how many boxes, the keyboard.

fgets keeps the Enter key

index	0	1	...	12	13	14
box	L	e	...	a	\n	\0

`Leyla Aliyeva` is 13 characters: boxes 0 to 12. Then `'\n'` in box 13, `'\0'` in box 14. That `\n` sent > to the next line.

scanf or fgets?

scanf("%s", s)	fgets(s, 50, stdin)
one **word**	a whole **line**
stops at a space	spaces too
no \n stored	keeps the \n

6 Loops over a string

Walk until the end mark

The string loop

Walk along the boxes until `'\0'`.

```
for (int i = 0; s[i] != '\0'; i++)
```

No size needed: the end mark says where to stop.

The recap loop, with `'\0'` in place of `0`.

The length of "cat"

```
char s[] = "cat";
int len = 0;

while (s[len] != '\0') {
    len++;
}
printf("length of %s: %d\n", s, len);
```

len	s[len]	!= '\0'?
0	'c'	yes

`len` starts at 0: the first box.

The length of "cat"

```
char s[] = "cat";
int len = 0;

while (s[len] != '\0') {
    len++;
}
printf("length of %s: %d\n", s, len);
```

len	s[len]	!= '\0'?
0	'c'	yes
1	'a'	yes
2	't'	yes
3	**'\0'**	**no: stop**

At box 3 the loop meets `'\0'` and stops. `len` is 3: it prints `length of cat: 3`.

Length as a function

```
int length(char s[]) {
    int len = 0;
    while (s[len] != '\0') {
        len++;
    }
    return len;
}
```

length of "Baku", "Programming", ""

```
4
11
0
```

No n: the `'\0'` ends the loop.

An int array and a string

class 10: int array

```
void print_array(int a[], int n)

print_array(a, 5);
```

today: a string

```
int length(char s[])

length(city);
```

> An `int` array needs its size. A string carries its own end, the `'\0'`.

Remove the Enter key

```
fgets(line, 50, stdin);
printf("length %d\n", length(line));

for (int i = 0; line[i] != '\0'; i++) {
    if (line[i] == '\n') {
        line[i] = '\0';
    }
}
printf("length %d: <%s>\n", length(line), line);
```

`'\n'` becomes `'\0'`.

Input `Leyla Aliyeva`: it prints `length 14`, then `length 13: <Leyla Aliyeva>`. The `\n` is gone.

Predict: count the vowels

```
int count_vowels(char s[]) {
    int count = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        char c = tolower(s[i]);
        if (c == 'a' || c == 'e' || c == 'i' ||
            c == 'o' || c == 'u') {
            count++;
        }
    }
    return count;
}
```

The call: `"Azerbaijan"`.

How many? Does A count?

Counting vowels

```
int count_vowels(char s[]) {
    int count = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        char c = tolower(s[i]);
        if (c == 'a' || c == 'e' || c == 'i' ||
            c == 'o' || c == 'u') {
            count++;
        }
    }
    return count;
}
```

`tolower`: A counts.

It prints `vowels in Azerbaijan: 5`.

Letters, digits, spaces

```
int letters = 0, digits = 0, spaces = 0;
for (int i = 0; line[i] != '\0'; i++) {
    if (isalpha(line[i])) {
        letters++;
    } else if (isdigit(line[i])) {
        digits++;
    } else if (line[i] == ' ') {
        spaces++;
    }
}
```

Input `Room B102, 16:00`: the program prints `letters 5, digits 7, spaces 2`. The comma and the `:` are none of the three.

Predict: to_upper

```
void to_upper(char s[]) {
    for (int i = 0; s[i] != '\0'; i++) {
        s[i] = toupper(s[i]);
    }
}

int main(void) {
    char s[] = "hello, world 2026";
    to_upper(s);
    printf("%s\n", s);
    return 0;
}
```

It returns nothing.

What does `main` print?

to_upper changes the string

```c
void to_upper(char s[]) {
    for (int i = 0; s[i] != '\0'; i++) {
        s[i] = toupper(s[i]);
    }
}

int main(void) {
    char s[] = "hello, world 2026";
    to_upper(s);
    printf("%s\n", s);
    return 0;
}
```

Output

```
HELLO, WORLD 2026
```

Like `double_all` in class 10.

Class 12: string.h

> **Length, copy, join, compare: the library does them.**

`strlen`, `strcpy`, `strcat`, `strcmp`

Today you wrote the loop that `strlen` hides.

Class 12 also turns `"2026"` into the number 2026.

7

Your turn

Two small string programs

Your turn: count_char

YOUR TURN

Write `int count_char(char s[], char c)`.

It returns how many times `c` is in `s`.

`count_char("banana", 'a')` is 3.

count_char: one answer

```
int count_char(char s[], char c) {
    int count = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] == c) {
            count++;
        }
    }
    return count;
}
```

"banana" with 'a', then 'z'

```
3
0
```

The string loop and a counter.

Your turn: backwards

YOUR TURN

Print a string backwards.

`"stressed"` gives `desserts`.

Hint: find the length first.

Backwards: one answer

```
char s[] = "stressed";
int len = 0;
while (s[len] != '\0') {
    len++;
}

for (int i = len - 1; i >= 0; i--) {
    printf("%c", s[i]);
}
printf("\n");
```

Output

```
desserts
```

From `len - 1` down to 0.

Today in one slide

1. A `char` is one character in single quotes, stored as a code
2. `c - 'a'` is a place; `c - '0'` is a digit's value
3. `<ctype.h>`: `isdigit`, `isalpha`, `toupper`…; yes is "not 0"
4. A string: `char` boxes ending in `'\0'`, printed with `%s`
5. `scanf("%s", s)` one word, no `&`; `fgets` a line with `\n`
6. Loop until `'\0'`: no size needed

Before Friday

1. **Knowledge check 11** on Blackboard: today's slides
2. **Quiz 3: Fri 16 October**, classes 7 to 10
3. Today's code: Blackboard, Week 6 module, **Class 11 code**

Next class: string.h, and turning numbers into strings and back.